

Diseño Basado en Agentes

Práctica 2. Movimiento de una agente en un mundo bidimensional



**UNIVERSIDAD
DE GRANADA**

Grado en Ingeniería Informática

Raúl Florentino Serra

Jesús Pereira Sánchez

Juan Manuel Guerrero Espigares

Índice

1	Introducción	2
2	Estructura de las clases	2
2.1	Mapa	2
2.2	Entorno	3
2.3	Sensores	4
2.4	Clases auxiliares	4
2.4.1	Clase Posición	4
2.4.2	Enumerado Movimientos	4
2.4.3	Comportamientos	5
2.5	Agente	5
2.5.1	Percepción	5
2.5.2	Decisión	5
2.5.3	Acción	5
2.5.4	Validación	6
2.5.5	Diagrama final	6
2.6	Interfaz Gráfica de Usuario	7
3	Decisión del Movimiento a realizar	7
3.1	Estrategia: LRTA*	7
3.2	Pasos en el algoritmo	8
3.2.1	Coste inicial: Aprendizaje	8
3.2.2	Bonificación de dirección	8
3.2.3	Bonificación de Momentum/Inercia	8
3.2.4	Desempate de posiciones	9
3.2.5	Aprendizaje	9
3.2.6	Establecimiento del movimiento decidido	9

1. Introducción

En esta práctica, se desarrolla un agente inteligente capaz de recorrer un mundo bidimensional, esquivando los obstáculos que tiene este mismo, hasta llegar a un objetivo. Para llevar esto a cabo, el agente sigue el clásico ciclo de *percepción* → *decisión* → *acción* → *validación*. Dentro de estos 4 comportamientos, el agente comunica con el entorno para poder percibir y actuar en función del estado del mundo en el que se encuentra.

Para poder ejecutar el movimiento del agente, se ha diseñado una arquitectura software que refleja el funcionamiento y protege perfectamente las tres grandes partes de este proyecto: **Agente**, **Entorno**, **Mundo**. Este diseño permite encapsular al mundo ante las acciones del agente mediante un entorno. Esto quiere decir que las acciones que realice el agente no se realizan sobre el mundo directamente, si no que el entorno las controla, procurando la coherencia de éstas mismas respecto al mapa y sus limitaciones.

Cabe destacar el uso de los sensores, los cuales son los que realmente encapsulan las acciones del agente ante el mundo. El agente posee un sensor, el cual se comunica con el entorno y por ende con el mapa. El entorno es el que realmente registra la posición del agente respecto del mapa.

2. Estructura de las clases

2.1. Mapa

Esta clase se encarga de ser la representación del mundo por el que el agente percibe y se mueve. La representación del mapa en la simulación es sencilla. Dada en un archivo de texto, las primeras líneas indican las dimensiones de este mismo. De la tercera línea en adelante, el mapa entero. Con 0 denotamos las casillas transitables, mientras que con -1 denotamos las que no lo son.

En nuestra clase **MapLoader** leemos de un archivo el mapa que queramos. Ya en memoria, se crea un mapa de las dimensiones que aparezcan en el archivo, y se lee una por una las componentes de esta matriz que están en el archivo de texto.

La clase **Mapa** es la que se encarga de gestionar esta matriz, implementando algunos métodos consultores de posiciones transitables, que hay en una determinada posición, dimensiones, etc. Además de métodos para emplazar objetos en el propio mapa. Las modificaciones se hacen en memoria en tiempo de ejecución y no se guardan en el archivo original, por lo que una vez finalizada la ejecución, no se modifica el mapa en ningún momento para futuras ejecuciones.

A continuación, se presenta el diagrama de clases del Mapa, junto a sus métodos y atributos.

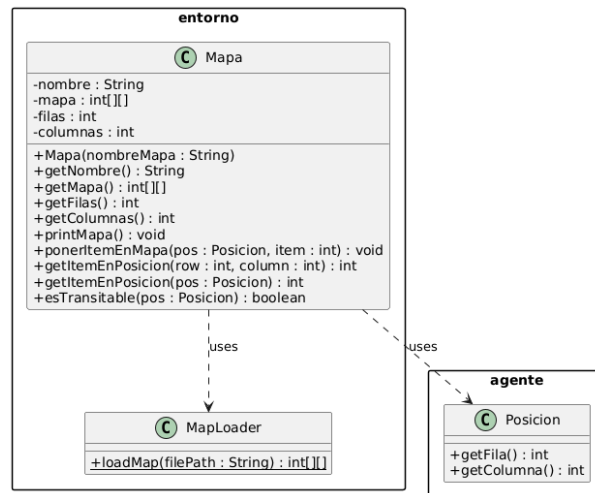


Figura 1: Diagrama de la clase Mapa

2.2. Entorno

La clase entorno se encarga de registrar el mapa y las posiciones del agente y del objetivo. Establece una interfaz para los sensores del agente con la que obtener el mapa y situar la posición del agente con cada acción que éste decida hacer, además de otros consultores.

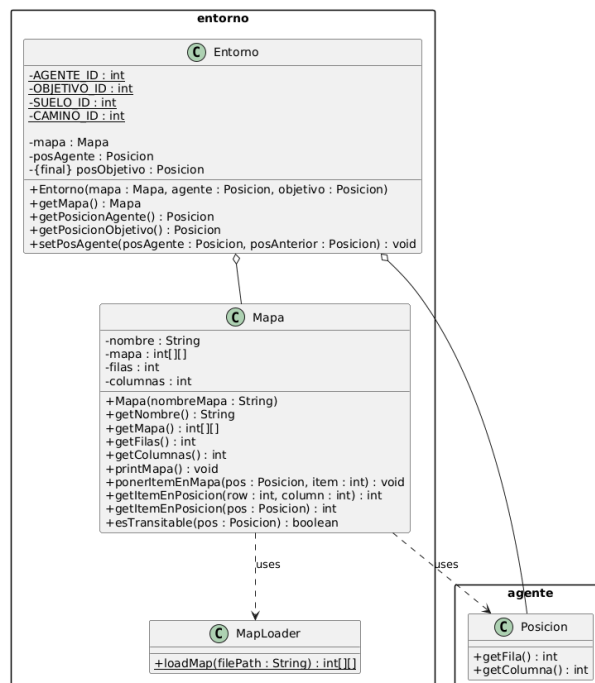


Figura 2: Diagrama del Entorno

2.3. Sensores

Aquí es dónde realmente se produce la encapsulación y se aseguran los movimientos que puede hacer el agente. El agente a la hora de consultar qué es lo que puede hacer, llama a su sensor (al método *verCasillasDisponibles()*), y este le devuelve una lista de movimientos posibles. En éste método, el sensor consulta privadamente el mapa. Se comprueba en las 4 casillas adyacentes a la posición del agente si son transitables o no, es decir, que en la matriz no haya un -1 y que además no sean casillas fuera de los límites del mapa. Para cada una de las casillas transitables adyacentes, hay un movimiento que se puede realizar para llegar a ellas. Al final, al agente se le devuelve una lista de **Movimientos** posibles. El tamaño máximo de esta lista será 4 (los 4 movimientos posibles, *UP*, *DOWN*, *LEFT*, *RIGHT*), y el mínimo 0 (en el peor de los casos en el que el agente se sitúe de alguna forma encerrado por muros).

Además, puede consultar en todo momento cuáles son las casillas vistas, un método útil a la hora de elegir qué movimiento será el que elija.

El diagrama de clase del Sensor es el siguiente:

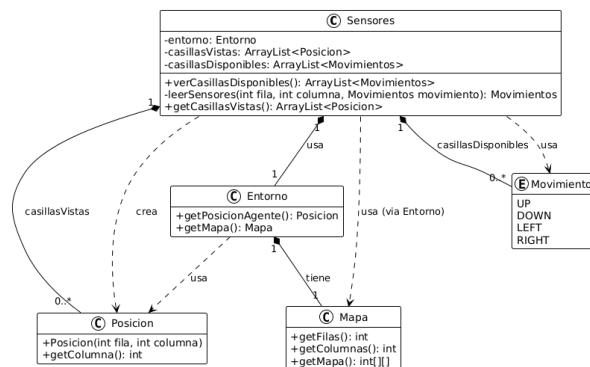


Figura 3: Diagrama de clase del Sensor

2.4. Clases auxiliares

Antes de explicar el funcionamiento del **Agente**, es necesario entender las clases auxiliares que facilitan su implementación y entendimiento.

2.4.1. Clase Posición

Una posición en nuestro **Mapa** son coordenadas cartesianas, *x* e *y*. Se han implementado métodos consultores. En el constructor, es cuando se define que valores toman en cada eje. Además, se ha sobrecargado el operador de "=", para poder comparar posiciones de forma más directa. También, un constructor de copia.

2.4.2. Enumerado Movimientos

Este enumerado contiene todos los movimientos que puede realizar un agente. Es importante resaltar que no son todos los movimientos que puede hacer el agente en un determinado momento, puesto que esto depende de la posición que ocupe en el mapa. Los movimientos son *UP*, *DOWN*, *LEFT*, *RIGHT*, *NONE*.

2.4.3. Comportamientos

Los 4 comportamientos de nuestro agente, *percepción* → *decisión* → *acción* → *validación*. La implementación de los comportamientos es sencilla, al final son llamadas a distintos métodos que el agente ejecuta. Estos comportamientos se ejecutan hasta que se encuentra el objetivo, esa es su condición de parada. Heredan de los *Behaviours de Jade*

2.5. Agente

El agente sabe que posición tiene y la de su objetivo. Además, tiene las casillas disponibles en cada momento de la toma de decisión gracias a los sensores. El agente guarda en su memoria las posiciones de las casillas que ha visitado. Además, también tiene un mapa de calor el cuál le ayudará a determinar la mejor ruta por dónde ha pasado y por dónde no.

En el método *setup()* que se hereda de la clase *Agent*, el agente añade sus respectivos comportamientos. Los comportamientos son **Percepción**, **DecisiónMov**, **HacerMov** (acción), **Validación**, como ya hemos comentado. Cada uno, tiene distintas delegaciones:

2.5.1. Percepción

El agente en la fase de *Percepción* debe de consultar los sensores para poder percibir el mundo. Para ello, tenemos, tenemos que consultar cuáles son los movimientos posibles que podemos hacer (**verCasillasDisponibles()**) y guardar en la memoria del agente qué es lo que ha visto **updateMemoriaVistas()**. Estos métodos forman parte del *action* del comportamiento.

Para ver los movimientos disponibles, usamos a los sensores, y para guardar en la memoria del agente lo que se ha visto, usamos la memoria local en el set de posiciones *memoriaVistas*.

La condición de parada de esta actividad es que hayamos encontrado el objetivo, por lo que al final será comprobar una condición booleana, en la que nuestra posición sea la del objetivo. En el **done()** llamamos al método del agente que comprueba lo mencionado.

2.5.2. Decisión

En este comportamiento, el agente debe de **decidir qué movimiento** hacer en función de la percepción que se tiene del mundo. Para ello, se llama al método del agente **decidirMov()**. Aquí vienen las distintas estrategias a implementar que se discutirán posteriormente. Al final, la decisión del movimiento más conveniente para la obtención del objetivo se guarda en la variable de tipo Movimiento, **mejorMovimiento**, usado por posteriores métodos.

La condición de parada es la misma que anteriormente: **haber llegado al objetivo**. Implementamos la misma lógica en el método **done()**.

2.5.3. Acción

El agente, habiendo decidido cuál es el mejor movimiento que realizar, **actúa en base al movimiento elegido**. Para ello, llama al método **hacerMov()**, el cual toma **mejorMovimiento** (elegido en la fase de Decisión), y le comunica al sensor que va a hacer ese movimiento, para que actualice en el entorno lo repercutiente al hacer esa acción. Además, el agente actualiza la memoria de las posiciones visitadas, su mapa de calor, el cual le ayudará a tomar decisiones en el futuro.

2.6. Interfaz Gráfica de Usuario

La interfaz de usuario se encarga de mostrar los movimientos del agente, el mundo que le rodea, y las posiciones de los elementos que lo conforman.

Para representar el mapa, nos valdremos de su naturaleza de matriz. En cada componente, encontramos al agente, un obstáculo, el objetivo, o nada (casillas transitables; suelo). Por cada componente que tengamos en la matriz, representamos visualmente ya sea con colores o con imágenes. Además de nuestra matriz, se implementa a la derecha un registro que se actualiza con cada movimiento del agente tomado, mostrando la posición, su batería y decisión y acción ejecutada. Nuestra interfaz es la siguiente;



Figura 5: Interfaz de usuario

3. Decisión del Movimiento a realizar

3.1. Estrategia: LRTA*

Durante la fase de decisión, el agente deberá elegir el movimiento más factible para poder llegar antes al objetivo. La estrategia que se ha implementado es la de **LRTA*** (*Learning Real Time A**). Este algoritmo se caracteriza por ser una variación del algoritmo A* que solo explora una parte limitada del mapa (en este caso el espacio que rodea al agente, la visión que los sensores le proporcionan). Además, cabe destacar su naturaleza **greedy**, en la que se elige el movimiento inmediato que más conviene. El movimiento que más conviene elegir se calcula con una fórmula de coste, el cual se asocia a las distintas posiciones del mapa. La fórmula sería:

$$f(s') = \text{costo}(s, s') + h(s')$$

Donde el costo de ir de s a s' es el incremento de energía, y h es la evaluación o estimación de la heurística en la posición s' . Antes de hacer el movimiento, el algoritmo debe de actualizar el coste del nodo actual. Esto se hace para poder actualizar el coste de la posición s , ya que su coste pudo haberse subestimado (calculado mal). Este aprendizaje se hace con el máximo entre el costo viejo y el costo recién calculado.

3.2. Pasos en el algoritmo

Dividimos el algoritmo de decisión LRTA implementado en distintas partes. Para cada movimiento que tiene el agente disponible en un instante antes de hacer una acción:

3.2.1. Coste inicial: Aprendizaje

Establecemos el coste de hacer ese movimiento. Primero de todo, calculamos el coste como la suma de la heurística aplicada a esa posición y el coste de energía por hacerlo.

La heurística (*getHeuristica(Posicion)*) que usaremos es la media entre la distancia Manhattan y la Euclídea de la posición actual a la posición objetivo. A la vez que usamos la función de la heurística, almacenamos para cada posición el valor heurístico aquellos valores de posición que no están guardados.

Si el coste de esta posición es menor al coste de aprendizaje que tenemos guardado, lo actualizamos. Esto nos servirá más tarde.

En este punto, habríamos completado la función

$$f(s') = \text{costo}(s, s') + h(s')$$

pero vamos a añadir más características para la toma de decisión.

3.2.2. Bonificación de dirección

Establecemos el **coste con penalización** de la próxima posición al aplicar el movimiento (*calcularCostePaso(PosiciónPróxima)*). Esto es aplicar la heurística a la próxima posición, y si la casilla se ha visitado, le aplicamos una penalización (peso mayor, más caro) en función de las veces en las que se ha estado ahí. De esta forma, castigamos a las casillas ya visitadas para que el agente recuerde por dónde ha ido, y no repita los mismos pasos.

Además, se calculan las direcciones en las que se encuentra el objetivo respecto a la posición del agente. Comparamos esta dirección (en signo) a la dirección entre la posición del agente y la posición siguiente (resultado de aplicar el movimiento que estamos analizando). En caso de ser la misma dirección (en alguno de los ejes), aplicamos una bonificación.

3.2.3. Bonificación de Momentum/Inercia

También se aplica un plus si el movimiento que estamos analizando es el mismo que hemos realizado anteriormente. De esta manera, calificamos positivamente el seguir haciendo el movimiento anterior, y tenemos un momentum. Esta estrategia nos servirá para poder recorrer los muros.

Al final, el coste de la decisión de esa casilla es el siguiente:

$$F_{\text{decisión}} = \text{costo}(s, s') + h(s') + P_{\text{dirección}} + P_{\text{momento}}$$

En resumen, calificamos el movimiento según si hemos pasado por ahí o no. La distancia Manhattan y Euclídea al objetivo. Si se mantiene la dirección al objetivo y el movimiento anterior prevalece sobre los distintos.

3.2.4. Desempate de posiciones

Cuando dos movimientos están empatados, es porque su coste es el mismo. Para compararlos seguramente (ya que son tipo double) los restamos, y si la diferencia es menor que 0.001, es porque son el mismo. Así nos evitamos fallos al comparar doubles, que son muy exactos.

Para aquellos movimientos cuyo valor de coste estén empatados (y que sean los mejores movimientos, es decir los que menor coste de decisión tienen hasta el momento), aquel movimiento que nos lleve a una casilla cuya distancia Euclídea sea menor será el elegido.

Si el movimiento es el menor que hemos encontrado hasta el momento, nos lo guardamos, ya que será el elegido.

3.2.5. Aprendizaje

Una vez se han evaluado todos los movimientos y almacenados en nuestra memoria de heurística, se procede a actualizar la heurística de la posición del agente actual. El valor de esta casilla es el máximo entre su valor heurístico y el valor f (coste + heurística) durante la evaluación de los posibles movimientos.

3.2.6. Establecimiento del movimiento decidido

Establecemos en la variable de la clase el movimiento que ha resultado como el mejor elegido para la fase de acción. Ya, el agente puede ejecutar o continuar con el siguiente comportamiento, el cual se encarga de ejecutar el mejor movimiento.